```
  protected $keyType = ‘string’;
}
```

**Composite Primary Key**

Each model must have at least one **ID** that can act as its primary key according to Eloquent. Eloquent models do not support **composite** main keys.

However, in addition to the main key that uniquely identifies the table, you are allowed to add other multi-column, unique indexes to your database tables.

## UUID and ULID Keys

You may decide to utilize UUIDs as your Eloquent model’s main keys rather than auto-incrementing integers. UUIDs are 36-character-long, globally unique alphanumeric identifiers.

Utilize the **`Illuminate\Database\Eloquent\Concerns\HasUuids`** trait on the model if you want a model to utilize a UUID key rather than an auto-incrementing integer key. Of course, you need to make sure that the model has a primary key column with a UUID equivalent:

```
use Illuminate\Database\Eloquent\Concerns\HasUuids;
use Illuminate\Database\Eloquent\Model;
class Chapter extends Model
{
  use HasUuids;
  // Code logic
}
$chapter = Chapter::create([‘title’ => ‘Chapter 1’]);
$chapter>id;
```

The **`HasUuids`** trait creates **ordered** UUIDs by default for your models. These UUIDs can be sorted lexicographically, making them more effective for index database storage:

By adding a **newUniqueId** function to a model, you can modify how a UUID is generated for that particular model. Additionally, you may define a **uniqueIds** function on the model to indicate which columns should get UUIDs: